



Pontifícia Universidade Católica do Rio Grande do Sul
Escola Politécnica

Projeto e Arquitetura de Software

Padrão Observer

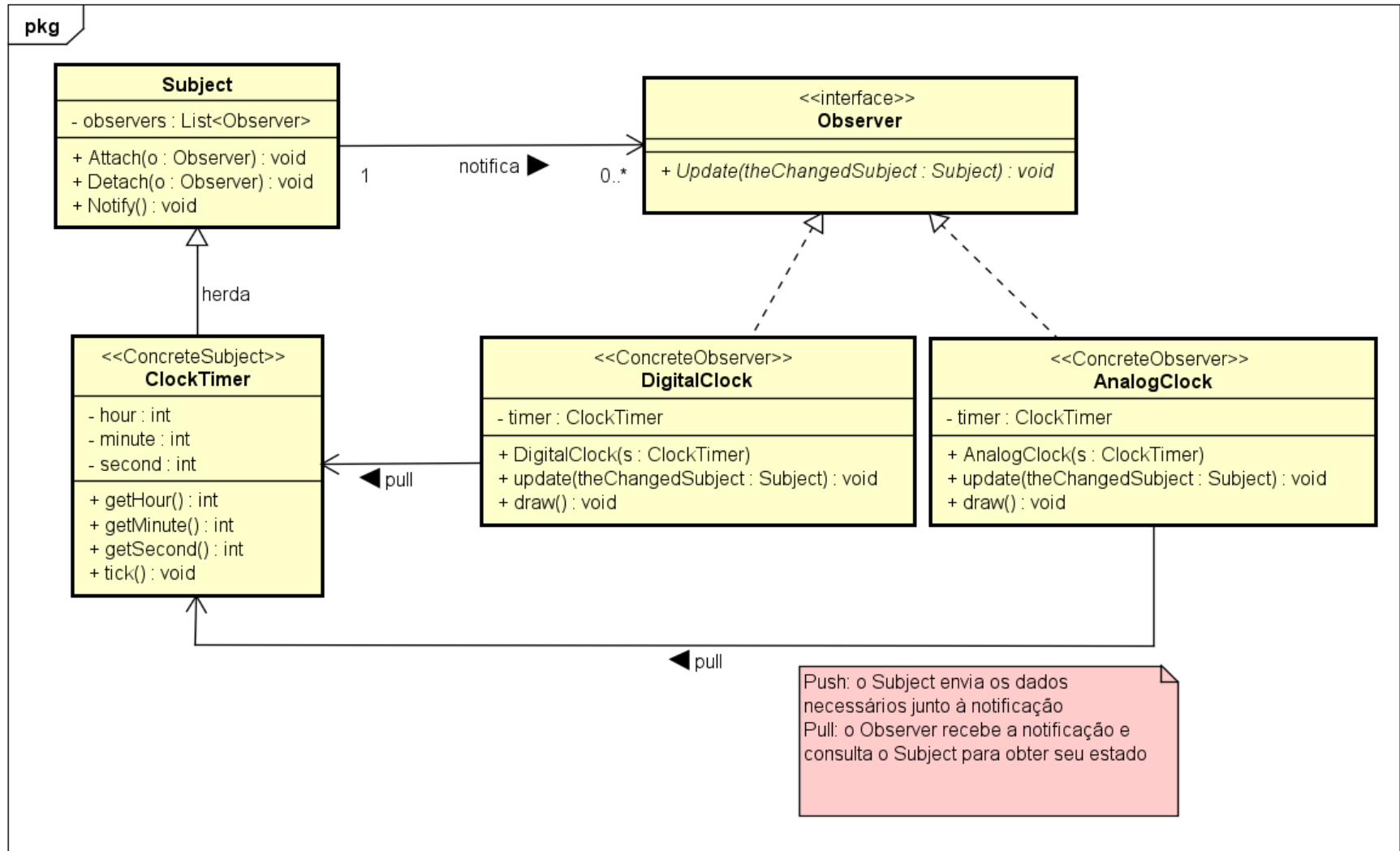
**Arthur Marques, Leonardo Ramos e
Maria Rita Rodrigues.**



- **Nome do padrão:** Observer
- **Propósito:** É um padrão comportamental que estabelece uma relação um-para-muitos utilizado quando vários objetos precisam acompanhar as mudanças de estado de outro objeto. Ele permite que essas atualizações ocorram automaticamente sem criar uma dependência direta entre as classes concretas. (GAMMA et al., 2002).
- **Escopo:** Objetos

- **Contexto:** Em muitos sistemas, diferentes objetos precisam permanecer consistentes quando o estado de um deles muda.
- **Problema:** O problema ocorre quando o objeto observado precisa conhecer diretamente as classes concretas de seus dependentes, criando forte acoplamento.
 - **Isso gera:**
 - alto acoplamento;
 - dificuldade para adicionar novos dependentes;
 - menor reutilização;
 - necessidade de modificar o objeto principal quando surgem novos consumidores.

- **Subject:**
 - Conhece seus Observers
 - Mantém uma lista de Observers
 - Attach() adiciona um Observer
 - Detach() remove um Observer
 - Notify() notifica os Observers
- **Observer:**
 - Define a interface Update() para os objetos que serão notificados
- **ConcreteSubject**
 - armazena o estado de interesse
 - notifica os Observers quando seu estado muda
- **ConcreteObserver**
 - mantém uma referência ao ConcreteSubject
 - implementa Update()
 - mantém seu estado consistente com o Subject



```
public class Subject {  
    private final List<Observer> observers = new ArrayList<>();  
  
    public void attach(Observer observer) {  
        observers.add(observer);  
    }  
  
    public void detach(Observer observer) {  
        observers.remove(observer);  
    }  
  
    public void notifyObservers() {  
        for (Observer observer : observers) {  
            observer.update(this);  
        }  
    }  
}
```

```
public class ClockTimer extends Subject {  
  
    private int hour;  
    private int minute;  
    private int second;  
  
    public ClockTimer(int hour, int minute, int second) {  
        this.hour = hour;  
        this.minute = minute;  
        this.second = second;  
    }  
    public int getHour() {return hour;}  
    public int getMinute() { return minute; }  
    public int getSecond() { return second;}  
  
    public void tick() {  
        second++;  
  
        if (second == 60) {  
            second = 0;  
            minute++;  
        }  
        if (minute == 60) {  
            minute = 0;  
            hour++;  
        }  
        if (hour == 24)  
            hour = 0;  
  
        notifyObservers();  
    }  
}
```

```
public interface Observer {  
    void update(Subject theChangedSubject);  
}
```

```
public class AnalogClock implements Observer {  
  
    private final ClockTimer timer;  
  
    public AnalogClock(ClockTimer timer) {  
        this.timer = timer;  
        timer.attach(this);  
    }  
  
    @Override  
    public void update(Subject theChangedSubject) {  
        if (theChangedSubject == timer) {  
            draw();  
        }  
    }  
  
    public void draw() {  
        System.out.printf(  
            format: "Relógio analógico atualizado: %02d:%02d:%02d\n",  
            timer.getHour(),  
            timer.getMinute(),  
            timer.getSecond()  
        );  
    }  
}
```

```
public class DigitalClock implements Observer {  
  
    private final ClockTimer timer;  
  
    public DigitalClock(ClockTimer timer) {  
        this.timer = timer;  
        timer.attach(this);  
    }  
  
    @Override  
    public void update(Subject theChangedSubject) {  
        if (theChangedSubject == timer) {  
            draw();  
        }  
    }  
  
    public void draw() {  
        System.out.printf(  
            format: "\nRelógio digital: %02d:%02d:%02d\n",  
            timer.getHour(),  
            timer.getMinute(),  
            timer.getSecond()  
        );  
    }  
}
```

```
import java.time.LocalDateTime;
public class Main {

    Run | Debug
    public static void main(String[] args) {

        // pega a hora atual do sistema
        LocalDateTime agora = LocalDateTime.now();

        // cria o ConcreteSubject com a hora atual
        ClockTimer timer = new ClockTimer(
            agora.getHour(),
            agora.getMinute(),
            agora.getSecond()
        );

        // cria e registra os ConcreteObservers
        DigitalClock digitalClock = new DigitalClock(timer);
        AnalogClock analogClock = new AnalogClock(timer);

        // mostra o horário inicial
        digitalClock.draw();
        analogClock.draw();

        // simula a passagem de 3 segundos
        timer.tick();
        timer.tick();
        timer.tick();
    }
}
```


Exemplo prático, implementação do chamado de uma função em um botão na web:

- Azul: Subject (o botão)
- Vermelho: implementação da inscrição em algum evento
- Verde: evento inscrito/observado
- Amarelo: callback function, o que o Observer vai fazer (neste caso ele é o próprio Observer)

```
button.addEventListener("click", () => {  
    console.log("clizou");  
});
```

• Observações:

– Pontos Positivos:

- Baixo acoplamento entre Subject e Observer;
- Subject e os Observers podem variar independentemente;
- Novos Observers podem ser adicionados sem alterar o Subject;
- Suporte a comunicação do tipo broadcast.

– Pontos Negativos:

- Uma mudança pode gerar diversas atualizações em cadeia;
- Podem ocorrer notificações desnecessárias;
- Pode ser difícil identificar o impacto de uma mudança;
- O protocolo de Update() nem sempre informar exatamente o que mudou.

• Referências:

- GAMMA, E. et al. *Padrões de projeto: soluções reutilizáveis de software orientado a objetos*. Porto Alegre: Bookman, 2002.